Docker Deployment Guide for phpIP

Deploy phpIP using Docker and Docker Compose with MySQL 8.0.

Prerequisites

- Docker (version 20.10+)
- Docker Compose (version 2.0+)
- Git

Quick Start

1. Clone and Configure

```
git clone -b internal-dev https://github.com/laosanlyu/phpip.git
cd phpip

# Copy the environment template
cp .env.example .env
```

> **Note:** The `-b internal-dev` flag clones the repository and checks out the `internal-dev` branch directly. This branch contains the Docker deployment setup.

2. Edit `.env` — Set Required Values

The following variables **must** be set. The stack will refuse to start without `DB_ROOT_PASSWORD`, `DB_USERNAME`, and `DB_PASSWORD`:

```
# Database — CHANGE THESE for production!
DB_HOST=mysql
DB_DATABASE=phpip
DB_USERNAME=phpip
DB_PASSWORD=your_secure_password
DB_ROOT_PASSWORD=your_secure_root_password

# Application
APP_URL=http://your-server-ip-or-domain
COMPANY_NAME="Your Company"

# Email (required for task notifications)
MAIL_HOST=smtp.gmail.com
MAIL_USERNAME=your-email@gmail.com
MAIL_PASSWORD=your-app-password
MAIL_FROM_ADDRESS=noreply@your-domain.com
MAIL_TO=admin@your-domain.com
```

> **APP_KEY:** Leave `APP_KEY=` empty. The entrypoint script will auto-generate a unique key on first startup and write it to `.env`. Do not fill in a placeholder value — the auto-generation only triggers when the value is exactly empty.

3. Build and Start

```
docker compose -f docker-compose.mysql.yml up -d --build
```

The entrypoint script will automatically:

- Install PHP dependencies (`vendor/`) if missing
- Generate `APP_KEY` if empty in `.env` (writes the key back to `.env`)
- Wait for MySQL to be ready
- Run database migrations (app container only)
- Cache config, routes, and views

4. Seed Reference Data (required for new installs)

Migrations create the tables but leave them empty. You must seed the reference data (countries, roles, event names, task rules) for the application to work.

Option A: Set `SEED_DATABASE=true` in the `app` service environment in `docker-compose.mysql.yml` (already present, default `false`). This auto-seeds on container start — safe to leave enabled as seeders use `insertOrIgnore` .

Option B: Run manually after the containers are up:

```
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed
```

5. Access the Application

- **Web application**: `http://YOUR_SERVER_IP` (default port 80)
- **phpMyAdmin**: `http://YOUR_SERVER_IP:8080` (requires MySQL login credentials)

The Nginx server accepts any hostname (`server_name _`), so no configuration change is needed when moving to a different server or domain.

Container Architecture

All services are defined in `docker-compose.mysql.yml` :

Container	Image	Purpose	Exposed Port
nginx	nginx:alpine	Web server, serves static files, proxies PHP to app	80, 443
app	phpip-app:latest (built)	PHP 8.4 FPM, runs Laravel application	Internal only (9000)
scheduler	phpip-app:latest (shared)	Runs `php artisan schedule:run` every 60 seconds	None
mysql	mysql:8.0	Database	Internal only
redis	redis:alpine	Cache and session store	Internal only
phpmyadmin	phpmyadmin/phpmyadmin	Database admin UI (login required)	8080

Container dependencies and startup order

```
mysql (healthy) ──→ app (healthy) ──→ nginx
                      │
                      ├──→ scheduler
                      │
                      └──→ phpmyadmin

redis (healthy)   (independent — no containers depend on it)
```

- **nginx** waits for `app` to be healthy before starting

- **app** waits for `mysql` to be healthy before starting
- **scheduler** waits for both `mysql` and `app` to be healthy
- **scheduler** reuses the same Docker image as `app` (no duplicate build)
- **redis** starts independently — no container declares a dependency on it, but the app uses it for cache/sessions once available

Volume mounts

Mount	Type	Purpose
`./:/var/www/html`	Bind mount	Project source code for app, nginx, scheduler
`./docker/php/php.ini`	Bind mount	PHP configuration
`./docker/nginx/*.conf`	Bind mount	Nginx configuration
`./docker/mysql/my.cnf`	Bind mount	MySQL configuration
`mysql_data`	Named volume	Persistent database data
`redis_data`	Named volume	Persistent cache data

Network

All containers communicate over `phpip-network` (Docker bridge). MySQL and Redis ports are **not exposed** to the host — they are only accessible between containers. This is intentional for security.

To access MySQL or Redis directly, use `docker exec`:

```
# MySQL shell
docker compose -f docker-compose.mysql.yml exec mysql mysql -u phpip -p phpip

# Redis CLI
docker compose -f docker-compose.mysql.yml exec redis redis-cli
```

How the Docker Build Works

The `docker/php/Dockerfile` uses a multi-stage build with three stages:

```
Stage: base          ← PHP 8.4 FPM + extensions + Composer
  ├── Stage: development   ← Adds Xdebug, full composer install (unused currently)
  └── Stage: production    ← --no-dev composer, builds assets, entrypoint
```

The compose file selects the production stage via `target: production`:

```
app:
  build:
    dockerfile: docker/php/Dockerfile
    target: production    # ← builds only base + production stages
```

The `development` stage is available in the Dockerfile for future use but is not currently referenced by any compose file.

Entrypoint Behavior

The entrypoint script (`docker/php/entrypoint.sh`) runs on every container start and behaves differently based on `CONTAINER_ROLE`:

Step	app container	scheduler container
Install vendor if missing	Yes	Yes
Generate APP_KEY if empty	Yes	Yes
Wait for MySQL	Yes	Yes
Run migrations	**Yes**	Skipped
Seed data (if env vars set)	**Yes**	Skipped
Cache config/routes/views	Yes	Yes
Start service	PHP-FPM	Schedule loop

Auto-seeding (optional)

Set these environment variables in `docker-compose.mysql.yml` under the `app` service to seed on startup:

```
environment:
  - SEED_DATABASE=true    # Seed reference data (countries, roles, event names, rules)
  - SEED_SAMPLES=true     # Seed example matters, actors, events (safe, uses insertOrIgnore)
```

Database Seeding

Seed Reference Data (Required for new installs)

```
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed
```

Alternative: Load Schema Dump

If you prefer loading a pre-built schema snapshot instead of running migrations + seeding (e.g. for faster setup or restoring a known-good state), you can load the schema dump **instead of** steps 3\u20134 above:

```
# Start only MySQL first
docker compose -f docker-compose.mysql.yml up -d mysql
# Wait for MySQL to be healthy, then load the schema
docker compose -f docker-compose.mysql.yml exec -T mysql \
  mysql -u root -p"YOUR_ROOT_PASSWORD" phpip < database/schema/mysql-schema.sql
# Then start all remaining services
docker compose -f docker-compose.mysql.yml up -d --build
```

> **Warning:** Do not run both migrations and the schema dump \u2014 they will conflict. Use one approach or the other.

Seed Sample Data (Optional)

```
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed --class=SampleSeeder
```

Individual Sample Seeders

```
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed --class=ActorSampleSeeder
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed --class=MatterSampleSeeder
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed --class=TaskSampleSeeder
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed --class=EventSampleSeeder
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed --class=ClassifierSampleSeeder
```

CSV Import/Rollback

Import data from a semicolon-delimited CSV file (see `database/seeders/import-template.csv` for the required format):

```
# Preview what would be imported (dry run, no changes)
docker compose -f docker-compose.mysql.yml exec app \
  php database/seeders/import-csv.php database/seeders/your-data.csv preview

# Import directly into the database
docker compose -f docker-compose.mysql.yml exec app \
  php database/seeders/import-csv.php database/seeders/your-data.csv direct
```

The `direct` mode inserts data and generates a JSON manifest file (e.g. `import-manifest-20260312_103000.json`) for rollback:

```
# Rollback a previous import using its manifest
docker compose -f docker-compose.mysql.yml exec app \
  php database/seeders/import-csv.php database/seeders/import-manifest-TIMESTAMP.json rollback
```

Available modes: `preview` (default), `seed` (write PHP array files), `direct` (insert into DB), `rollback` (undo a direct import).

Fresh Start (destroys all data)

```
docker compose -f docker-compose.mysql.yml exec app php artisan migrate:fresh --seed
docker compose -f docker-compose.mysql.yml exec app php artisan db:seed --class=SampleSeeder
```

> **Warning:** `migrate:fresh` drops all tables. Never use in production with real data!

Backup and Restore

Automated Backups

A backup script is provided at `scripts/backup-db.sh`. It creates compressed MySQL dumps, verifies them, and automatically cleans up backups older than 30 days.

```
# Run manually
./scripts/backup-db.sh

# Or specify a custom backup directory
./scripts/backup-db.sh /opt/backups/phpip
```

Backups are saved to `backups/` as `phpip_YYYYMMDD_HHMMSS.sql.gz`.

Schedule Periodic Backups

Add a cron job to run backups automatically:

```
crontab -e


# Daily at 2:00 AM
0 2 * * * /path/to/phpip/scripts/backup-db.sh >> /path/to/phpip/backups/cron.log 2>&1
```

Restore from Backup

```
gunzip < backups/phpip_20260304_020000.sql.gz | \
  docker compose -f docker-compose.mysql.yml exec -T mysql \
  mysql -u root -p"YOUR_ROOT_PASSWORD" phpip
```

Manual Backup (without script)

```
docker compose -f docker-compose.mysql.yml exec -T mysql \
  mysqldump -u root -p"YOUR_ROOT_PASSWORD" --single-transaction --routines --triggers phpip \
  | gzip > backup.sql.gz
```

Migrating to Another Server

Steps

```
# === On the NEW server ===

# 1. Clone the repository (internal-dev branch)
git clone -b internal-dev https://github.com/laosanlyu/phpip.git
cd phpip

# 2. Copy .env from the old server (contains APP_KEY, passwords)
scp old-server:/path/to/phpip/.env .env

# 3. Copy the latest database backup
scp old-server:/path/to/phpip/backups/latest.sql.gz .

# 4. Copy any uploaded files
scp -r old-server:/path/to/phpip/storage/app/ storage/app/

# 5. Build and start
docker compose -f docker-compose.mysql.yml up -d --build

# 6. Restore the database
gunzip < latest.sql.gz | docker compose -f docker-compose.mysql.yml exec -T mysql \
  mysql -u root -p"YOUR_ROOT_PASSWORD" phpip

# 7. Verify
curl http://localhost
```

Important: Copy the `.env` file from the old server — the `APP_KEY` must match or encrypted data and sessions will break.

Common Commands

```
# View logs
docker compose -f docker-compose.mysql.yml logs -f app
docker compose -f docker-compose.mysql.yml logs -f nginx

# Run artisan commands
docker compose -f docker-compose.mysql.yml exec app php artisan migrate
docker compose -f docker-compose.mysql.yml exec app php artisan config:cache

# Access MySQL shell
docker compose -f docker-compose.mysql.yml exec mysql mysql -u phpip -p phpip

# Rebuild after code/Dockerfile changes
docker compose -f docker-compose.mysql.yml down
docker compose -f docker-compose.mysql.yml build app
docker compose -f docker-compose.mysql.yml up -d

# Stop all containers
docker compose -f docker-compose.mysql.yml down

# Stop and remove all data (database, redis)
docker compose -f docker-compose.mysql.yml down -v
```

Optional Configurations

Set these in `.env`. Not required for basic deployment.

EPO Patent Services (OPS)

For automatic patent family import. Register at https://developers.epo.org/

```
OPS_APP_KEY=your_key
OPS_SECRET=your_secret
```

Renewr API

For renewal management:

```
RENEWR_API_URL=https://api.renewr.io/...
RENEWR_API_KEY=your_key
```

SharePoint Integration

For document management:

```
SHAREPOINT_ENABLED=true
SHAREPOINT_CLIENT_ID=your_id
SHAREPOINT_CLIENT_SECRET=your_secret
```

Production Checklist

- [ ] `APP_ENV=production` and `APP_DEBUG=false` in `.env`

- [ ] Strong database passwords set (`DB_PASSWORD` , `DB_ROOT_PASSWORD`)
- [ ] `APP_URL` set to your domain or IP
- [ ] SMTP email configured for task notifications
- [ ] Database backups scheduled (see Backup and Restore)
- [ ] Firewall configured (only ports 80/443 need external access)

Troubleshooting

Container Status

```
docker compose -f docker-compose.mysql.yml ps
docker compose -f docker-compose.mysql.yml logs app
docker compose -f docker-compose.mysql.yml logs mysql
```

Common Issues

Issue	Cause	Solution
Stack won't start with error about env vars	Missing `.env` or required DB variables not set	Ensure `.env` exists with `DB_ROOT_PASSWORD` , `DB_USERNAME` , `DB_PASSWORD`
502 Bad Gateway	PHP-FPM not ready yet	Wait 30 seconds — nginx waits for app health check
"MySQL is unavailable" loop	Wrong DB credentials or MySQL still starting	Verify `DB_*` values in `.env` , check `docker logs phpip-mysql`
Missing vendor folder	Normal on fresh clone	Entrypoint auto-runs `composer install`
No CSS/JS styling	`public/build/` missing	Should exist from git; if not, run: `docker compose -f docker-compose.mysql.yml exec app sh -c "npm install && npm run build"`

Permission Errors

```
docker compose -f docker-compose.mysql.yml exec app \
  chown -R www-data:www-data /var/www/html/storage /var/www/html/bootstrap/cache
```

Clear Caches

```
docker compose -f docker-compose.mysql.yml exec app php artisan config:clear
docker compose -f docker-compose.mysql.yml exec app php artisan cache:clear
docker compose -f docker-compose.mysql.yml exec app php artisan route:clear
docker compose -f docker-compose.mysql.yml exec app php artisan view:clear
```

Database Connection Errors

- Ensure `DB_HOST=mysql` in `.env`

- Wait 30–60 seconds for MySQL first-time initialization

Schema Loading Fails with SSL Error

Load directly from the MySQL container:

```
docker compose -f docker-compose.mysql.yml exec -T mysql \
  mysql -u root -pYOUR_ROOT_PASSWORD phpip < database/schema/mysql-schema.sql
```

Project Files Reference

File	Purpose
`docker-compose.mysql.yml`	Main compose file (MySQL 8.0, production)
`.env.example`	Environment template — copy to `.env`
`docker/php/Dockerfile`	Multi-stage PHP 8.4 FPM image (base/development/production)
`docker/php/entrypoint.sh`	Container startup script (migrations, seeding, caching)
`docker/php/php.ini`	PHP production config (OPcache on, errors off)
`docker/php/php-dev.ini`	PHP development config (Xdebug, errors on) — unused currently
`docker/nginx/default.conf`	Nginx site configuration
`docker/nginx/nginx.conf`	Nginx main configuration (gzip, performance)
`docker/mysql/my.cnf`	MySQL configuration (utf8mb4, InnoDB tuning)
`scripts/backup-db.sh`	Automated database backup with rotation

Support

- phpIP Wiki
- Report Issues